In the name of Allah

Data Networks

Prof. Mohammad Reza Pakravan

Final Project

Farzan Aghaei 401101202

Sharif University of Technology

Electrical Engineering Department

Contents

1 Introduction **2**

2 Core Functionality **2**

2.1 Class Initialization . . . 3
2.2 Server Functionality Management . . . 3
2.2.1 start method . . . 3
2.2.2 stop method . . . 4
2.2.3 accept_connections method . . . 4
2.2.4 parse_request method . . . 5
2.2.5 handle_client method . . . 5
2.2.6 url blocking methods . . . 7
2.2.7 cache methods . . . 7
2.2.8 run_traceroute method . . . 8

3 GUI Implementation **8**

3.1 Class Initialization . . . 8
3.2 Style Configuration . . . 9
3.3 Widget Creation . . . 10
3.4 App Icon . . . 11
3.5 Status Color Update . . . 11
3.6 Core Integration Methods . . . 11
3.6.1 Proxy Control Methods . . . 11
3.6.2 Logging System . . . 12
3.6.3 Blocklist and Caching Management . . . 12
3.6.4 Traceroute Implementation . . . 13

4 Testing and Validation **13**

4.1 Start and Stop . . . 13
4.2 URL Blocking . . . 15
4.3 Caching . . . 17
4.4 Traceroute . . . 17
4.5 Log . . . 18

1 Introduction

A proxy server acts as an intermediary between clients (web browsers) and destination servers. Instead of clients connecting directly to servers, they connect to the proxy server, which forwards requests and returns responses. This architecture provides several benefits:

- **Content Filtering**: Block access to specific URLs
- **Performance Improvement**: Cache frequently accessed resources
- **Traffic Monitoring**: Log and analyze network requests
- **Security**: Add an additional layer between clients and servers

In this project I implemented a comprehensive HTTP proxy server with the following features:

- Complete HTTP proxy functionality with TCP socket handling
- URL blocking with customizable lists
- Response caching for performance optimization
- Traceroute utility for network diagnostics
- Modern GUI for real-time monitoring
- Threaded architecture for concurrent request handling
- Status indicators and statistics

The proxy server was developed in Python using socket programming, threading, and Tkinter. The following sections provide a detailed explanation of both the core functionality and user interface implementation.

2 Core Functionality

The core functionality of the proxy server is implemented in a class named `HTTPProxyServer`. This class handles all network operations, request processing, and management features using socket and threading modules.

The methods in this class are:

- log
- start
- stop
- accept_connections
- parse_request
- handle_client
- is_blocked
- load_blocked_urls
- clear_blocked_urls
- get_cached_url
- clear_cache
- run_traceroute

In the following parts of this section I will explain what each of these methods do and the functionality of the core.

2.1 Class Initialization

```python
class HTTPProxyServer:
    def __init__(self, port=8080):
        self.port = port  # Listening port for proxy
        self.running = False  # Server state flag
        self.server_socket = None  # Server socket
        self.cache = {}  # Cache storage
        self.blocked_urls = set()  # URLs to block
        self.lock = threading.Lock()  # Thread synchronization lock
        self.log_callback = None  # Logging callback function
        self.request_count = 0  # Total requests count
        self.blocked_count = 0  # Blocked requests count
        self.cache_hits = 0  # Cache hits count
```

Listing 1: HTTPProxyServer Class Initialization

The initialization method sets up the proxy's base components:

- `port`: The TCP port on which the proxy listens for client connections (default: 8080)
- `running`: Boolean flag indicating whether the server is currently active
- `server_socket`: The main socket object that accepts incoming connections
- `cache`: Dictionary that stores cached responses with URLs as keys
- `blocked_urls`: Set containing URLs that should be blocked
- `lock`: Threading lock to ensure thread-safe access to shared resources
- `log_callback`: Function reference for logging messages to the GUI
- Statistics counters: Track request count, blocking activity, and cache performance

2.2 Server Functionality Management

The server working cycle is controlled through the mentioned methods, which manage the server's operation.

2.2.1 start method

```python
    def start(self):
        if self.running:
            return False
        self.running = True

        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.server_socket.bind(('127.0.0.1', self.port))
        self.server_socket.listen(8)

        self.log(f"Proxy server started on port {self.port}", "success")
        threading.Thread(target=self.accept_connections).start()
        return True
```

Listing 2: Server Start Method

The `start()` method performs the following operations:

- Checks if the server is already running to prevent duplicate starts
- Creates a TCP socket using IPv4 addressing and stream-oriented protocol
- Sets socket option to allow address reuse for quick restarts

- Binds the socket to localhost (127.0.0.1) and the specified port
- Sets the socket to listening mode with a backlog of 8 pending connections
- Starts a thread running `accept_connections()` to handle incoming connections
- Logs the start event
- Returns True on success and False on failure

2.2.2 stop method

```
    def stop(self):
        if not self.running:
            return

        self.running = False
        if self.server_socket:
            self.server_socket.close()
        self.log("Proxy server stopped")
```

Listing 3: Server Stop Method

The `stop()` method performs the following operations:

- Checks if the server is running to avoid unnecessary operations
- Updates the running state flag to False
- Closes the server socket to stop accepting new connections
- Logs the shutdown event

2.2.3 accept_connections method

```
    def accept_connections(self):
        while self.running:
            try:
                client_socket, addr = self.server_socket.accept()
                self.request_count += 1
                threading.Thread(target=self.handle_client,
                                 args=(client_socket, addr)).start()

            except OSError:
                break
```

Listing 4: Connection Acceptance Loop

The `accept_connections()` method runs in a dedicated thread and performs:

- Continuous loop while the server is running
- Accepts incoming client connections using `accept()` function in socket
- Increments the total request counter
- Starts a new thread for each client connection to handle requests
- Catches `OSError` exceptions that occur when the socket is closed during shutdown

The new thread passes the requests to `handle_client()` method.

The `OSError` exception is added because the application kept giving an irrelevant OS error when the socket was closed, So I used try-except structure to break the loop when the error was generated (when the socket was closed).

2.2.4 parse_request method

```
    def parse_request(self, request):
        lines = request.split('\r\n')
        method, url, http_version = lines[0].split()

        headers = {}
        for line in lines[1:]:
            if not line:
                break
            if ': ' in line:
                key, value = line.split(': ', 1)
                headers[key] = value

        return method, url, http_version, headers
```

Listing 5: HTTP Request Parsing

The HTTP request and response structure is illustrated in the figure below.

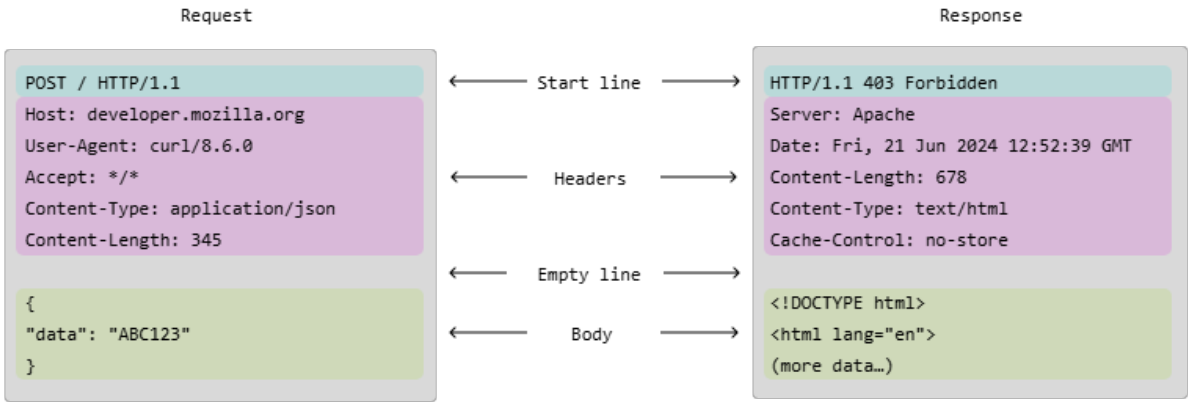


Figure 1: The structure of HTTP request and response messages

The request parser:

- Splits the request into individual lines
- Extracts the HTTP method, URL, and version from the request line (start line)
- Parses headers into key-value pairs (the header is kept in a dictionary variable)
- Handles header continuation and proper delimiter detection
- Returns method, url, http_version and headers

Using this method I managed to break the requests into their components and separate them.

2.2.5 handle_client method

```
    def handle_client(self, client_socket, addr):
        try:
            request = client_socket.recv(1024).decode()
            if not request:
                return

            # Parse HTTP request
            method, url, http_version, headers = self.parse_request(request)

```

```
            # Extract host and path
            host = headers.get('Host', '')
            if not host:
                return

            self.log(f"Request #{self.request_count}: {method} {url} from {addr[0]}",
                "request")

            # Check if URL is blocked
            if self.is_blocked(url):
                self.blocked_count += 1
                self.log(f"Blocked URL: {url}", "blocked")
                response = (
                "HTTP/1.1 403 Forbidden\r\n"
                "Content-Type: text/html\r\n\r\n"
                "<html><body style='background: #1e1e2e; color: #cdd6f4; font-family:
                    sans-serif; padding:  40px; text-align: center;'>"
                "<h1 style='color: #f38ba8;'>403 Forbidden</h1>"
                "<p style='font-size: 1.2em;'>This URL has been blocked by the proxy
                    server.</p>"
                "</body></html>"
                )
                client_socket.sendall(response.encode())
                return

            # Check cache
            cached_response = self.get_cached_url(url)
            if cached_response:
                self.cache_hits += 1
                self.log(f"Serving from cache: {url}", "cache")
                client_socket.sendall(cached_response)
                return

            # Forward request to destination server
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
                server_socket.connect((host, 80))
                server_socket.sendall(request.encode())
                response = b""
                while True:
                    response = server_socket.recv(32768)
                    client_socket.sendall(response)
                    break

            # Cache the response
            if method == "GET" and (response.startswith(b"HTTP/1.1 200") or
                response.startswith(b"HTTP/1.1 304")):
                if url not in self.cache:
                    self.cache[url] = response
                    self.log(f"Cached: {url}", "cache")

        except Exception as e:
            self.log(f"Error: {str(e)}", "error")

        finally:
            client_socket.close()
```

Listing 6: Client Request Handling

The request handling process performs the following operations:

- Receives and decodes the client request
- Parses the HTTP request into its components

- Checks for the existence of the Host header
- Logs the request details
- Checks if the URL is blocked and responds with 403 Forbidden if so
- Checks the cache for existing responses
- Forwards the request to the destination server
- Streams the response back to the client
- Caches successful GET responses

2.2.6 url blocking methods

```
    def is_blocked(self, url):
        with self.lock:
            return any(blocked_url in url for blocked_url in self.blocked_urls)

    def load_blocked_urls(self, filename):
        with open(filename) as f:
            with self.lock:
                self.blocked_urls = {line.strip() for line in f if line.strip()}
        self.log(f"Loaded {len(self.blocked_urls)} blocked URLs", "success")
        return

    def clear_blocked_urls(self):
        with self.lock:
            self.blocked_urls.clear()
        self.log("Cleared blocked URLs")
```

Listing 7: URL Blocking Implementation

The URL blocking system features:

- `is_blocked()`: Checks if URL contains any blocked substring
- `load_blocked_urls()`: Loads URLs from a text file
- `clear_blocked_urls()`: Clears blocked URL list
- Thread-safe operations using locking
- Partial URL matching (block "example.com" blocks subdomains and paths)

2.2.7 cache methods

```
    def get_cached_url(self, url):
        with self.lock:
            return self.cache.get(url)

    def clear_cache(self):
        with self.lock:
            self.cache.clear()
        self.log("Cache cleared")
```

Listing 8: Caching Implementation

Caching mechanism:

- Dictionary-based storage with URL as key and response as value
- Thread-safe access using locking

- Only caches successful GET responses (HTTP 200 and HTTP 304)
- Cache persists until explicitly cleared

2.2.8 run_traceroute method

```
    def run_traceroute(self, host):
        # Determine the correct command for the OS
        if platform.system().lower() == "windows":
            command = ['tracert', '-d', '-h', '5', host]
        else:
            command = ['traceroute', '-m', '5', host]

        result = subprocess.run(command, capture_output=True, text=True).stdout
        return result
```

Listing 9: Traceroute Implementation

Traceroute features:

- OS detection for command compatibility
- Runs the traceroute command using the subprocess module
- Captures the output streams
- Returns formatted output

3 GUI Implementation

The graphical user interface is implemented in the `ProxyGUI` class, providing visual control and monitoring for the proxy server. Built using Tkinter and ttk widgets, it features a modern dark theme with responsive components and real-time statistics.

3.1 Class Initialization

```
class ProxyGUI:
    def __init__(self):
        self.proxy = ProxyServer()
        self.proxy.log_callback = self.log_message

        # Create main window
        self.root = tk.Tk()
        self.root.title("HTTP Proxy Server")
        self.root.geometry("960x640")
        self.root.configure(bg="#1e1e2e")
        self.root.minsize(900, 600)

        # Configure styling
        self.style = ttk.Style(self.root)
        self.style.theme_use("clam")
        self.configure_styles()

        # Create widgets
        self.create_widgets()

        # Set app icon
        self.set_icon()

        # Set initial status color to red (disconnected)
        self.update_status_color()
```

Listing 10: ProxyGUI Class Initialization

Key initialization elements:

- `proxy`: Instance of the core ProxyServer
- `log_callback`: Connects core logging to GUI display
- Main window configuration: Dark theme (#1e1e2e) with dimensions 960x640 and minimum size constraints 900x600
- Style Configuration: Uses 'clam' theme with custom dark mode adaptations
- Widget Creation: Creates the needed widgets
- App Icon: Sets up a custom icon for the application
- Initial Proxy Status Color: Sets the initial proxy status color to red (disconnected)

3.2 Style Configuration

```
    def configure_styles(self):
        # Configure colors
        bg_color = "#1e1e2e"
        fg_color = "#cdd6f4"

        self.style.configure(".", background=bg_color, foreground=fg_color,
                             font=("Inter", 10))
        self.style.configure("TFrame", background=bg_color)
        self.style.configure("TLabel", background=bg_color, foreground=fg_color)
        self.style.configure("TButton", background="#45475a", foreground=fg_color,
                             borderwidth=1, focuscolor="#89b4fa", padding=6)
        self.style.map("TButton", background=[("active", "#5c5e7e"), ("pressed", "#313244")],
                       foreground=[("active", fg_color)])
        self.style.configure("TEntry", fieldbackground="#313244", foreground=fg_color,
                             insertcolor=fg_color, padding=5)
        self.style.configure("TCombobox", fieldbackground="#313244", foreground=fg_color)
        self.style.configure("TNotebook", background=bg_color, borderwidth=0)
        self.style.configure("TNotebook.Tab", background="#313244", foreground=fg_color,
                             padding=[10, 5], font=("Inter", 10, "bold"))
        self.style.map("TNotebook.Tab", background=[("selected", bg_color),
                       ("active", "#45475a")], foreground=[("selected", "#89b4fa")])
        self.style.configure("Vertical.TScrollbar", background="#45475a", arrowcolor=fg_color)

        # Configure tags for log text
        self.log_tags = {
            "info": {"foreground": "#89b4fa"},
            "error": {"foreground": "#f38ba8"},
            "success": {"foreground": "#a6e3a1"},
            "request": {"foreground": "#cba6f7"},
            "cache": {"foreground": "#f9e2af"},
            "blocked": {"foreground": "#f38ba8"},
            "system": {"foreground": "#a6e3a1"}
        }
```

Listing 11: configure_styles method

The custom style configuration:

- Establishes consistent color scheme for dark mode UI
- Configures base styles for all widgets
- Defines color tags for log message categorization
- Ensures visual coherence across interface components

3.3 Widget Creation

```
    def create_widgets(self):
        # Create tabbed interface
        notebook = ttk.Notebook(self.root)

        # Control Panel Tab
        control_tab = ttk.Frame(notebook)
        notebook.add(control_tab, text="Control Panel")

        # Server Control Section
        control_frame = ttk.LabelFrame(control_tab, text="Server Management")
        control_frame.pack(fill=tk.X, padx=15, pady=15)

        # Port configuration
        ttk.Label(control_frame, text="Proxy Port:").grid(row=0, column=0)
        self.port_var = tk.StringVar(value="8080")
        port_entry = ttk.Entry(control_frame, textvariable=self.port_var, width=8)
        port_entry.grid(row=0, column=1)

        # Control buttons
        self.start_btn = ttk.Button(control_frame, text="Start",
        command=self.start_proxy)
        self.start_btn.grid(row=0, column=2, padx=5)

        self.stop_btn = ttk.Button(control_frame, text="Stop",
        command=self.stop_proxy, state=tk.DISABLED)
        self.stop_btn.grid(row=0, column=3, padx=5)

        # Statistics display
        stats_frame = ttk.Frame(control_frame)
        stats_frame.grid(row=0, column=4, sticky="e")
        ttk.Label(stats_frame, text="Requests:").pack(side=tk.LEFT)
        self.req_var = tk.StringVar(value="0")
        ttk.Label(stats_frame, textvariable=self.req_var).pack(side=tk.LEFT)

        # Activity Log Tab
        log_tab = ttk.Frame(notebook)
        notebook.add(log_tab, text="Activity Log")

        # Log display
        self.log_text = scrolledtext.ScrolledText(
        log_tab, bg="#313244", fg=fg_color,
        font=("Consolas", 10), wrap=tk.WORD
        )
        self.log_text.pack(fill=tk.BOTH, expand=True)

        # Configure log colors
        for tag, style in self.log_tags.items():
        self.log_text.tag_config(tag, **style)
```

Listing 12: Main Widgets Construction

GUI component structure:

- Tabbed notebook interface for organized functionality (Control panel and Activity log)
- Control panel: Port configuration, command buttons, traceroute and statistics
- Real-time statistics: Total/Blocked/Cached request counters with live updating
- Activity log: Color-coded scrolling results and stats area

3.4 App Icon

```
    def set_icon(self):
        myAppID = 'arbitrary string'
        ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID(myAppID)
        self.root.iconbitmap("F:\\term6\data_network\project\\python\proxy.ico")
```

Listing 13: set_icon method

Using this method I managed to set a custom icon for my proxy app.

3.5 Status Color Update

```
    def update_status_color(self):
        if self.proxy.running:
            self.status_label.configure(foreground="#a6e3a1")  # Green when connected
        else:
            self.status_label.configure(foreground="#f38ba8")  # Red when disconnected
```

Listing 14: update_status_color method

This method uses the running boolean element of the proxy server to update the color of the status label. The label will be green if connected and red if not.

3.6 Core Integration Methods

3.6.1 Proxy Control Methods

```
    def start_proxy(self):
        try:
            port = int(self.port_var.get())
            if not (1024 <= port <= 65535):
                raise ValueError("Port must be between 1024 and 65535")

            self.proxy.port = port
            if self.proxy.start():
                self.status_var.set(f"Status: Running on port {port}")
                self.start_button.config(state=tk.DISABLED)
                self.stop_button.config(state=tk.NORMAL)
                self.log(f"Proxy started on port {port}", "success")
                self.update_status_color()
            except Exception as e:
                self.log(f"Failed to start proxy: {str(e)}", "error")
                messagebox.showerror("Error", f"Failed to start proxy:\n{str(e)}")

        def stop_proxy(self):
            self.proxy.stop()
            self.status_var.set("Status: Not running")
            self.start_button.config(state=tk.NORMAL)
            self.stop_button.config(state=tk.DISABLED)
            self.log("Proxy server stopped")
            self.update_status_color()
```

Listing 15: Proxy Control Functions

Control flow features:

- Input validation for port number
- State management for control buttons
- Error handling while starting the proxy

- Updating the status color
- Logging the status changes

3.6.2 Logging System

```
    def log(self, message, tag="info"):
        self.log_text.config(state=tk.NORMAL)
        self.log_text.insert(tk.END, message + "\n", tag)
        self.log_text.config(state=tk.DISABLED)
        self.log_text.see(tk.END)

        # Update statistics
        self.request_count_var.set(str(self.proxy.request_count))
        self.blocked_count_var.set(str(self.proxy.blocked_count))
        self.cache_hits_var.set(str(self.proxy.cache_hits))

    def log_message(self, message, tag="info"):
        self.log(message, tag)
```

Listing 16: Logging Implementation

Logging system characteristics:

- Automatic scrolling to newest entries
- Color coding based on message type
- Real-time statistics updating
- State management for text widget

3.6.3 Blocklist and Caching Management

```
    def load_blocked_urls(self):
        filename = filedialog.askopenfilename(title="Select blocked URLs file",
                                              filetypes=[("Text files", "*.txt"),
                                              ("All files", "*.*")])
        if filename:
            self.proxy.load_blocked_urls(filename)
            self.log(f"Loaded {len(self.proxy.blocked_urls)} blocked URLs", "success")
            self.log("Blocked URLs:")
            for i in self.proxy.blocked_urls:
                self.log(f"{i}")
            messagebox.showinfo("Success",
                                f"Loaded {len(self.proxy.blocked_urls)} blocked URLs")

    def clear_blocked_urls(self):
        self.proxy.clear_blocked_urls()
        self.log("Cleared blocked URLs")
        messagebox.showinfo("Success", "Blocked URLs cleared!")

    def clear_cache(self):
        self.proxy.clear_cache()
        self.log("Cache cleared")
        messagebox.showinfo("Success", "Cache cleared!")
```

Listing 17: Blocklist Handling

URL blocking and Caching management features:

- File dialog for blocklist text file selection
- Clear blocked urls list and cache storage when needed

- Visual confirmation of actions in the activity log

3.6.4 Traceroute Implementation

```
    def run_traceroute(self):
        host = self.trace_add.get().strip()
        if not host:
            messagebox.showwarning("Warning", "Please enter a hostname!!!")
            return

        self.log(f"Running traceroute to {host} ...")
        result = self.proxy.run_traceroute(host)
        messagebox.showinfo("Success", "Traceroute Completed!")

        # Display in log
        self.log(f"{result}", "system")
```

Listing 18: run_traceroute method

Traceroute features:

- Running the traceroute command properly
- Notifying with a message box when the traceroute is finished
- Results display in activity log with proper formatting

4 Testing and Validation

Now it's time to test the functionality of the proxy server app I built.

We should check all the operations and also the error situations and observe that the results are accurate. This way we make sure that the proxy app is working properly and everything is OK.

In the following sections of this part, I investigate the following operations.

Test Case	**Procedure**
Start and Stop	Configure port and browser to use proxy, access http sites, make sure everything is loaded completely
URL Blocking	Load URLs block list, attempt access, clear list and attempt again
Caching	Access same page multiple times and observe the caching process
Concurrent Access	Multiple clients access in the same time
Traceroute	Run traceroute to various hosts and observe results
Log	check whether the event logging is done correctly

Table 1: Functional Test Cases

4.1 Start and Stop

First, the browser network setting should be configured. I used Mozilla Firefox. To do this, I went into the Network Settings tab to change the connection setting. Then I selected Manual proxy configuration and put 127.0.0.1 and 8080 in HTTP Proxy and Port boxes. The configuration is shown in the figure in the beginning of the next page.

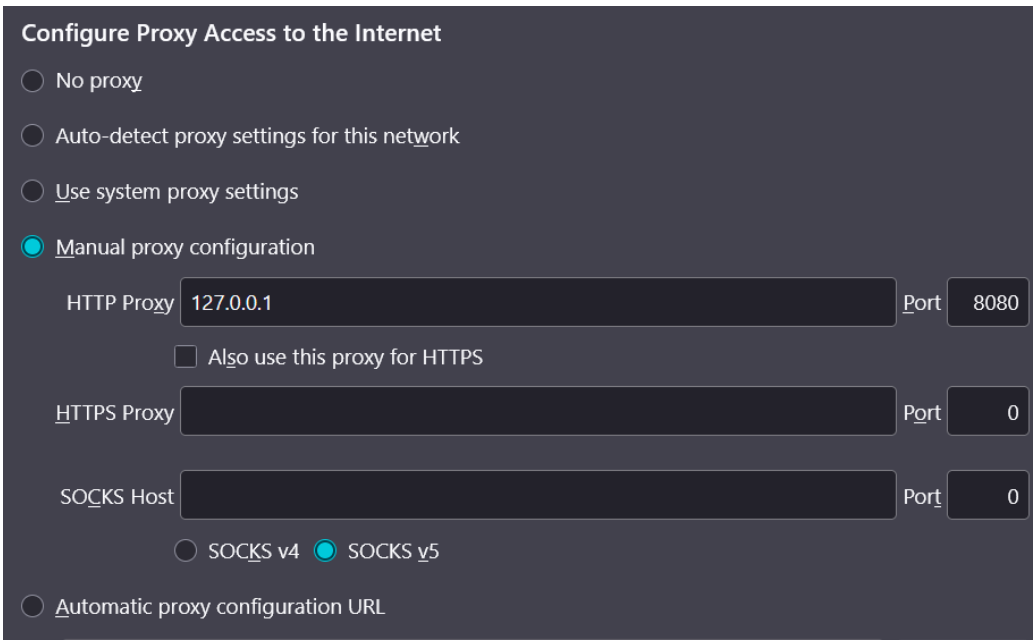


Figure 2: The browser connection configuration

After setting up the browser I compiled my python code and the application started. The main window of the app is presented in figure 3.

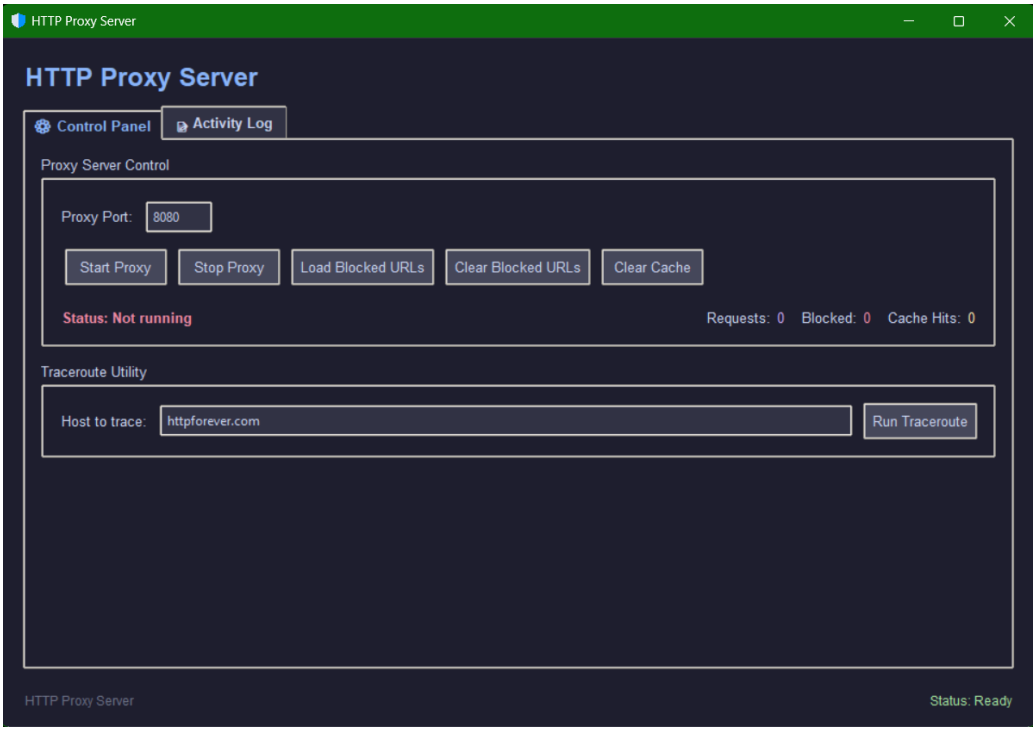


Figure 3: The main window the proxy app

Now I hit the Start Proxy button to start the proxy. The picture of the window after pressing start is demonstrated in figure in the next page. Notice that the status label changed from a red **Not running** into a green **Running on port 8080**.

Lets try to access an http website with and without the proxy server turned on. I used the website httpforever to check the functioning of my proxy.

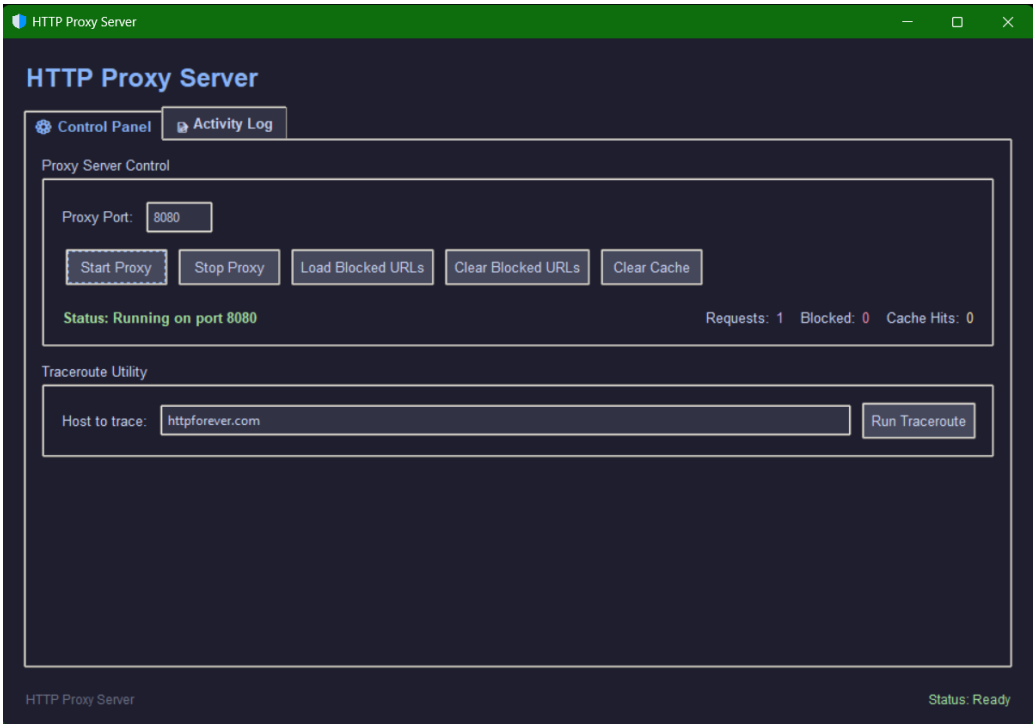


Figure 4: The main window after hitting start

You can see the results below. As expected the website is accessed using the proxy server and denied when the proxy is disconnected.

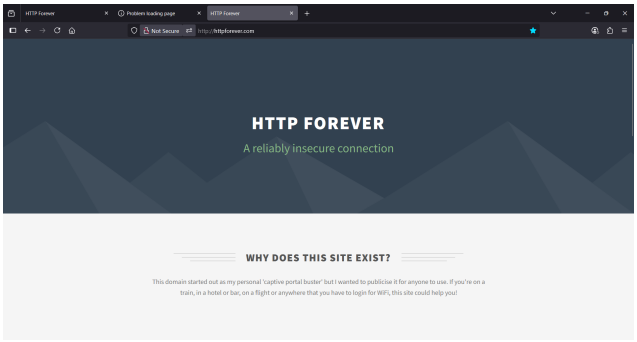


Figure 5: Proxy Connected

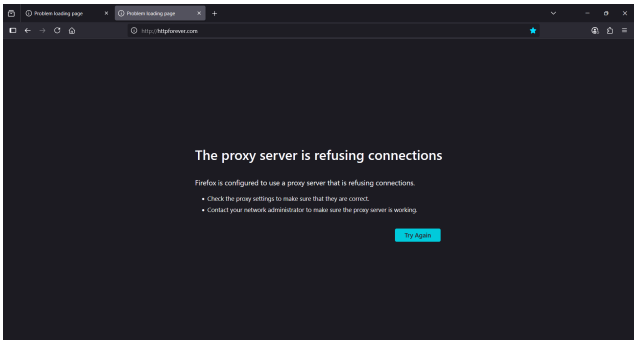


Figure 6: Proxy Disconnected

Lets try the scenario that the port number is less than 1024. I set the port number to 808 and pressed the start button.

It is observed that an error message pops up mentioning that the port number is not valid and the value must be greater than 1024. You can see this result in the figure in the beginning of the next page.

4.2 URL Blocking

There is a text file provided to import as the list of the blocking urls named blocked_urls. I hit the Load Blocked URLs button and then selected the mentioned file. Then a pop-up appears saying "Loaded 10 blocked URLs" with the title success. Now to see if this really works and the urls in the text file are blocked correctly I accessed httpforever again. I got a **403 Forbidden** text as a result which is exactly the result we were expecting. You can see the screenshots demonstrating this process in figures 17, 18 and 10.

The activity log tab and the event logs after each operation are all displayed in the Log section 4.5.

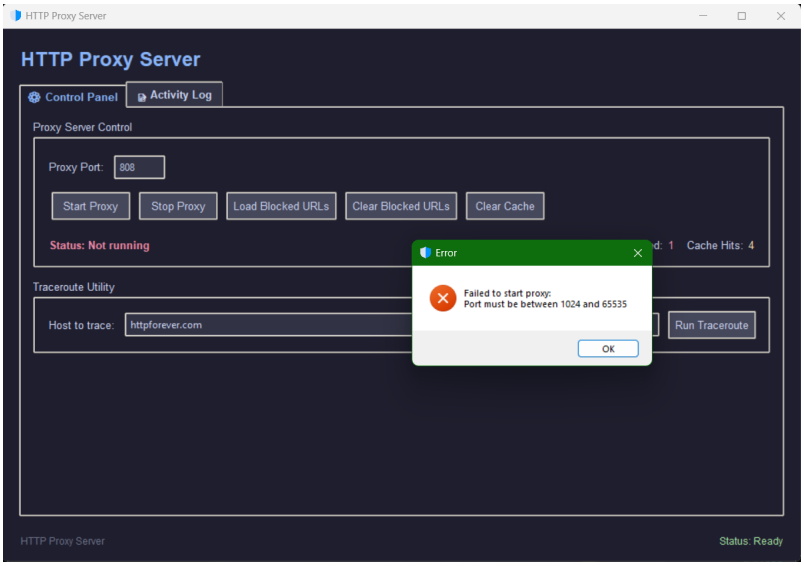


Figure 7: Port number less than 1024

Now to test the correct operation of the Clear Blocked URLs function, I accessed the website again after clicking on the Clear Blocked URLs button. As it was expected, the website loaded properly and without any trouble because the url got out of the black list. It is worth noting that after clicking the button a message showed up telling me that the Blocked URLs cleared!

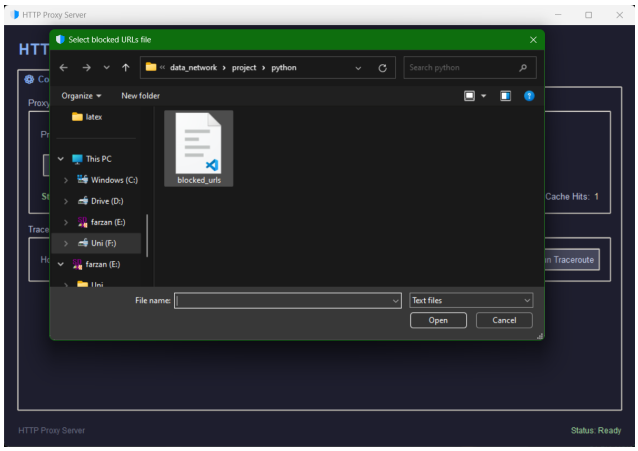


Figure 8: Selecting the text file containing the list of blocked urls

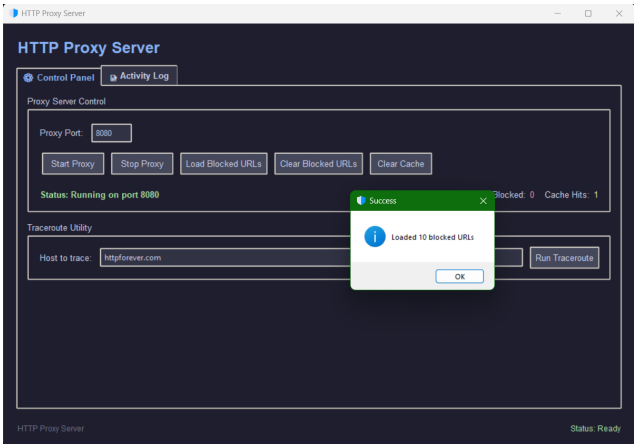


Figure 9: The pop-up showing that the blocked urls were loaded successfully

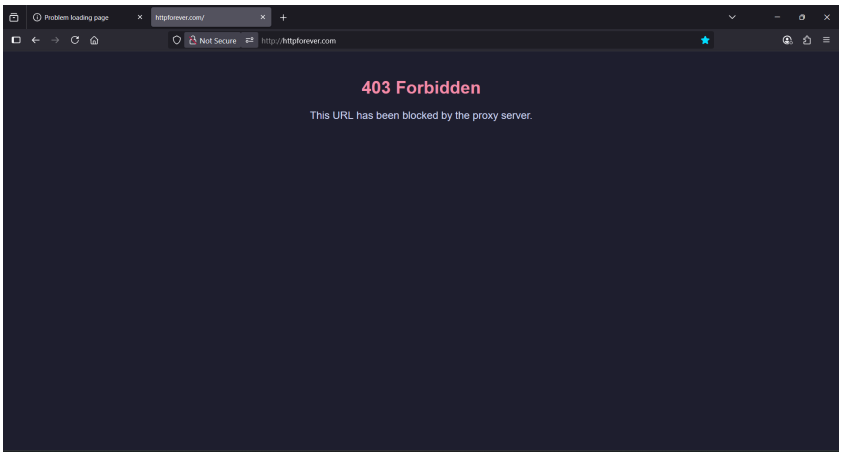


Figure 10: The result of accessing a blocked url in the browser

4.3 Caching

Now we get to the caching functionality of the proxy server. Each accepted and valid request is saved in the cache storage eventually. After the first attempt to access a website the response gets cached and in the next access attempts the response is send faster because it is forwarded from the cache storage. To test this property I first accessed the test website and then refreshed the page. The content of the page loaded quicker than the first time because the response was saved in the proxy's cache storage and was given to the client immediately.

Then we need to check the Clear Cache button and see if it is functioning accurately. I hit the button and got a "Cache cleared!" message in an info box. Then I accessed the site again and since the cache storage was empty and the saved response was removed, the content of the web page took a little bit longer to load.

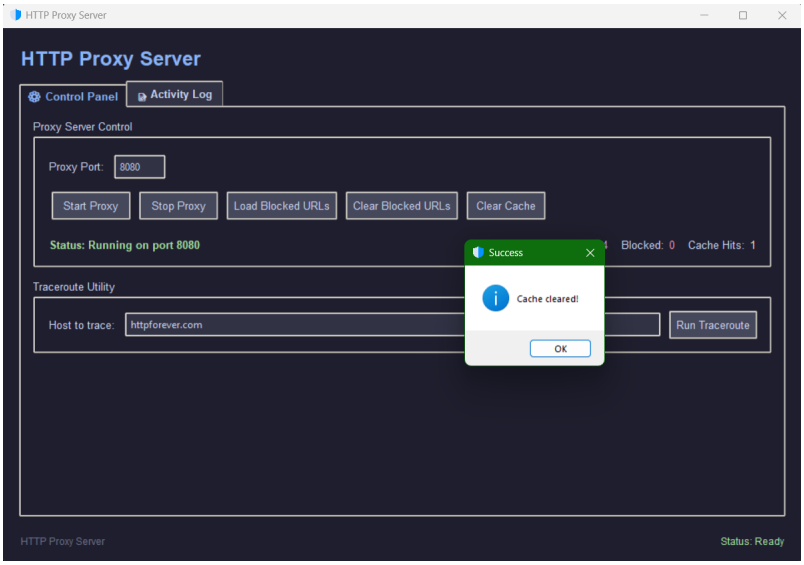


Figure 11: The Cache cleared pop-up message

4.4 Traceroute

It is time to test the interesting option of our proxy app. The traceroute command. The default value in the host url box is the address (url) of the website we used to test our proxy app's capabilities and functions (httpforever). I pressed the run button and waited for a few seconds for the process to finish. Again like the other parts a pop-up message showed up telling me that the traceroute process is completed and I can observe the results in the Activity Log tab.

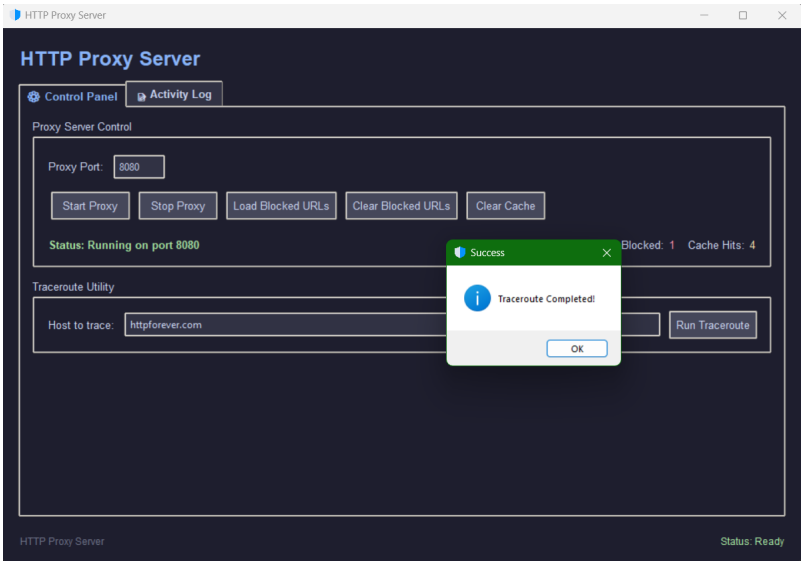


Figure 12: The Traceroute Completed pop-up message

4.5 Log

In this section we observe the log messages in the Activity Log tab that appear when something happens. It can be seen in the following figures the screenshots of the activity log before and after the mentioned functions.

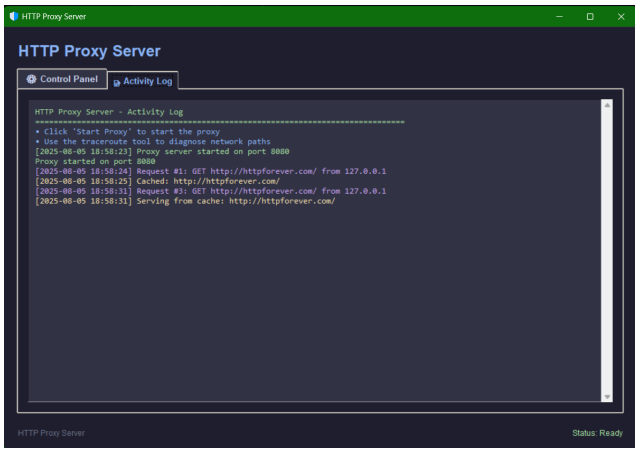


Figure 13: Access request to the test site, caching process and the result of access to a cached requests

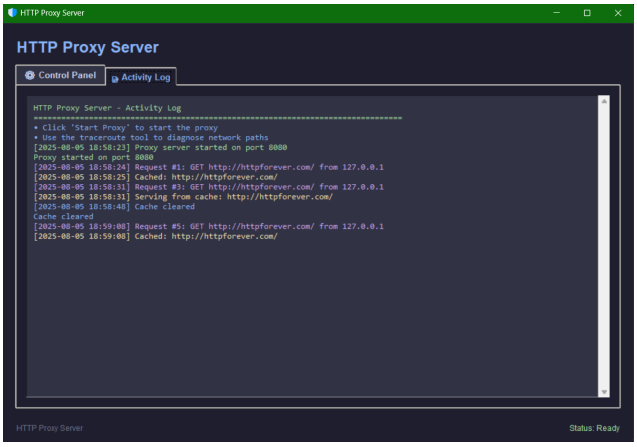


Figure 14: Clear cache operation message log and the correct operation of the function

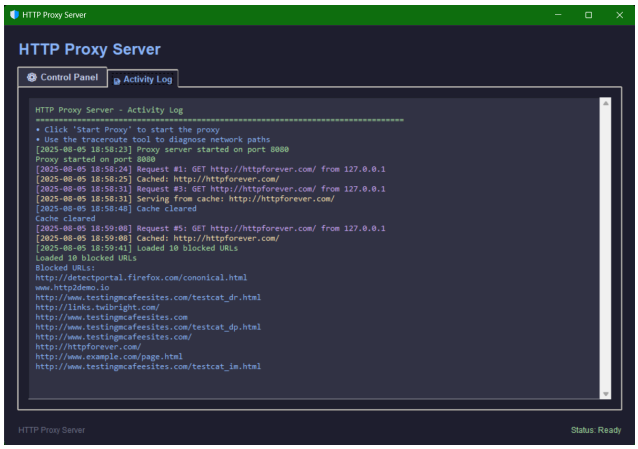


Figure 15: Load Blocked URLs function result and the list of the blocked urls in the loaded file

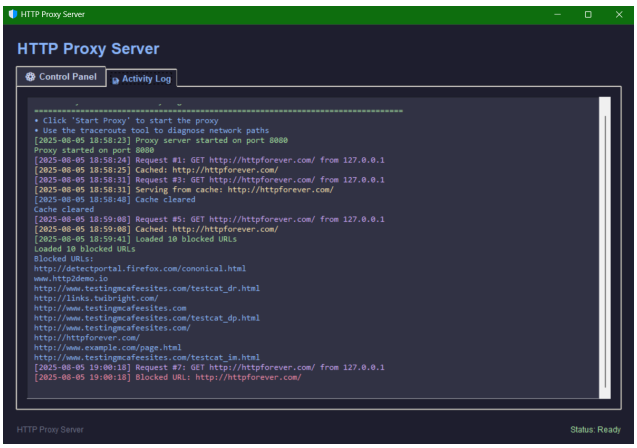


Figure 16: Requests to blocked urls get blocked by the proxy and a proper message is displayed in the log

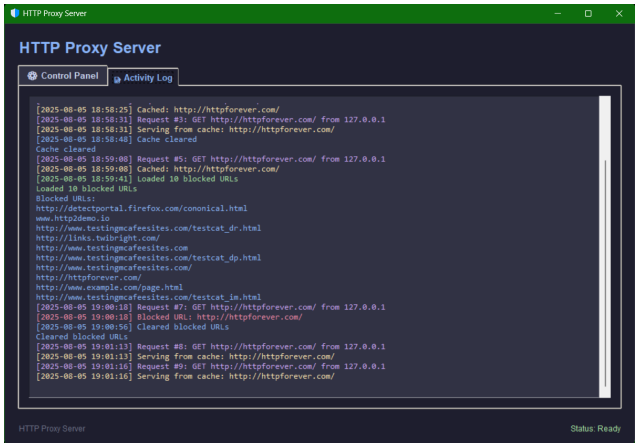


Figure 17: After deleting the blocked urls list the requests are handled without any problem like before

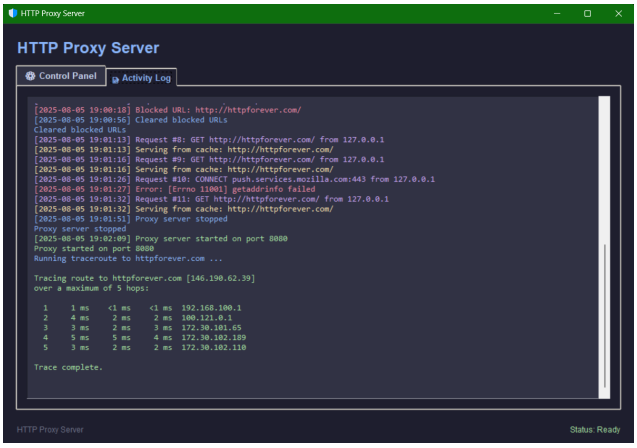


Figure 18: You can see the results from the traceroute process in the activity log